

CCA REPORT

AI Sentinel: Smart Traffic Monitoring and Surveillance System

Data Mining (CT-356)



NED University of Engineering and Technology
Department of Computer Science and Information Technology

Submitted by:

Amna Altaf (DT-23003)

Sheikh Asma Javed (DT-23027)

Under the Guidance of:

Sir Rohail Qamar

Session: 2023 – 2027

RUBRICS

Criteria and Scales				
Satisfactory (2)	Average (1)		Unsatisfactory (0)	
Criterion 1: Use of Resources: To what extent has the student has used resources? (CA1: Range of resources)				
The student has effectively used wide variety of resources	The student has somewhat effectively used some resources		The student has failed to use variety of resources	
Criterion 2: Data Quality: To what extent has the student improved data quality by? (CA2: Level of Interactions)				
The student has successfully improved data quality by applying data preprocessing techniques	The student has somewhat improved data quality by applying data preprocessing techniques		The student has failed to improve data quality by applying data preprocessing techniques	
Criterion 3: Data Analysis: To what extent has the student performed Data Analysis? (CA3: Familiarity)				
The student has successfully analyzed data using aggregation, graphs and tables	The student has somewhat analyzed data using aggregation, graphs and tables		The student has failed to analyze data using aggregation, graphs and tables	
Criterion 4: Model(s) Selection: To what extent has the student selected the model? (CA5: Familiarity)				
The student has applied model(s) that fits with the domain problem	The student has applied model(s) that somewhat fits with the domain problem		The student has applied model(s) that does not fit with the domain problem	
Criterion 5: System Usability: To what extent has the student used creative use of knowledge? (CA3: Innovation)				
The student has successfully demonstrated and evaluated data mining techniques	The student has somewhat demonstrated and evaluated data mining techniques		The student has not demonstrated data mining and evaluated techniques	
Project Title:				
Group Members Roll #				
Total Marks:	_____/10		Teacher's Signature:	

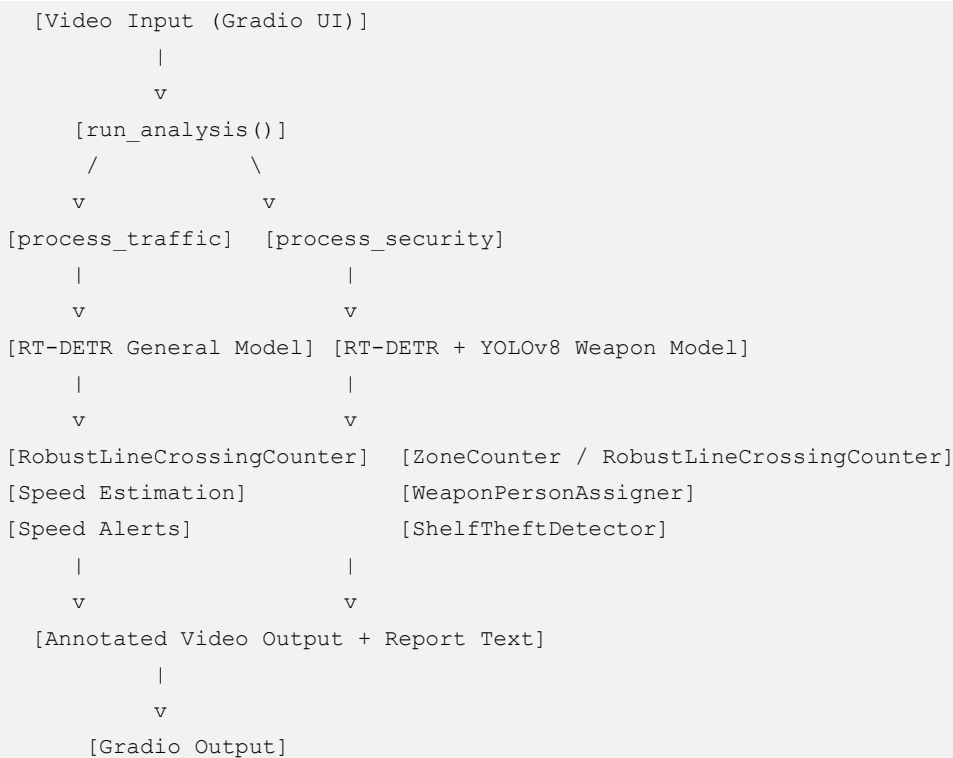
1. System Design and Logic Flow

1.1 Architecture Overview

The system is a dual-mode AI video analytics application. It accepts a video input through a Gradio web interface and routes it to one of two processing pipelines: Traffic Monitoring or Surveillance. Both pipelines share the same model infrastructure but apply different detection classes, counting strategies, and alerting logic.

1.2 Architecture Diagram

The diagram below shows how the major modules interact:



1.3 Data Flow

Data enters the system as a raw video file. It flows through the following stages:

- A video file is uploaded through the Gradio interface and its path is passed to `run_analysis()`.
- `run_analysis()` checks the selected mode and calls either `process_traffic()` or `process_security()`.
- Each frame is decoded using OpenCV (`cv2.VideoCapture`). The frame is fed into the RT-DETR general model, which returns bounding boxes with track IDs via ByteTrack.
- In Traffic Mode, box centroids are passed to `RobustLineCrossingCounter` to count unique vehicles crossing a horizontal band, and per-frame pixel displacement is converted to speed in km/h using a real-world scale factor.

- In Surveillance Mode, frames are additionally processed by the YOLOv8 weapon model every 5 frames. WeaponPersonAssigner links weapon detections to the nearest tracked person. ShelfTheftDetector monitors items in a defined shelf zone and raises a theft alert if an item disappears after a person was nearby.
- Annotated frames are written to a temporary .mp4 output file using cv2.VideoWriter. A text report is assembled from counting and alert data, and both are returned to the Gradio interface.

1.4 Algorithm Rationale

YOLOv8 for Weapon Detection

YOLOv8 (You Only Look Once, version 8) is a single-stage object detector that performs detection in one forward pass of a convolutional neural network. A custom YOLOv8m model is trained on a labeled gun dataset for the single class 'weapon'. The medium variant (yolov8m) is chosen as a balance between inference speed and accuracy. The model is fine-tuned from the pretrained yolov8m.pt checkpoint using transfer learning, with 50 epochs and image size 640. An optimal confidence threshold is derived post-training by computing the F1 score across the precision-recall curve and selecting the threshold that maximizes F1, clamped to the range [0.30, 0.85].

RT-DETR for General Object Detection

RT-DETR (Real-Time Detection Transformer, large variant) is used for general object detection because its transformer-based architecture provides strong performance on COCO classes without the anchor tuning required by YOLO. It is loaded pre-trained on COCO-80 and is not fine-tuned. It handles detection of persons, vehicles, and bag categories.

ByteTrack for Multi-Object Tracking

ByteTrack is used as the tracking backend (passed as tracker='bytetrack.yaml' to Ultralytics). ByteTrack assigns persistent integer track IDs to detections across frames using a Kalman filter and the Hungarian algorithm for IoU-based matching. This enables reliable per-object counting, speed estimation, and weapon-to-person assignment.

Speed Estimation

Speed is estimated by computing the Euclidean distance in pixels between a tracked object's centroid across consecutive frames, multiplying by a real-world scale factor (meters per pixel), multiplying by frame rate, and converting to km/h. A rolling average over 7 frames smooths the estimate. Results exceeding 180 km/h are discarded as noise.

Robust Line-Crossing Counting

A horizontal counting band of 60 pixels total width (line_y +/- 30 px) is defined at the vertical midpoint of the frame. An object is counted when its centroid transitions from one side of the band to the other. A ghost memory retains the last known side of vanished tracks for 8 frames to handle brief detection gaps. Unconfirmed detections (without a track ID) are also counted if their centroid falls within the band and their bounding box does not overlap any previously counted box.

2. Module-Level Breakdown

The entire system is implemented in a single Jupyter notebook. The following describes the primary classes and functions in order of the critical execution path.

2.1 Utility Functions

get_label(cls)	Maps a COCO class integer to a human-readable string. Classes 24, 26, 28 are unified under 'bag'.
box_center(box)	Returns the (x, y) centroid of a bounding box given as (x1, y1, x2, y2).
iou(b1, b2)	Computes Intersection over Union between two bounding boxes. Used for detection fusion and duplicate suppression.
fuse_detections(dets, iou_thresh)	Non-maximum suppression over weapon detections. Sorts by confidence and keeps only detections with IoU below threshold with already-kept boxes.
draw_box(frame, box, label, conf, color)	Draws a labeled bounding box on a frame with a filled label background.
draw_box_with_speed(frame, box, label, conf, speed_kmh, color)	Same as draw_box but appends speed in km/h to the label tag.
overlay_alerts(frame, alert_lines, footer)	Writes alert text lines in red at the top-left of the frame, and an optional footer line at the bottom.
draw_counts(frame, counts, w, h)	Writes per-class count totals at the bottom-right of the frame.

2.2 RobustLineCrossingCounter

Counts unique objects that cross a horizontal or vertical line band. This is the primary counting mechanism for both Traffic Mode and the second iteration of Surveillance Mode.

Input	line_coord: float (pixel position of line), axis: 'y' or 'x'
Key state	prev_side: dict mapping track ID to last side (-1 or +1). counted_ids: set of already-counted IDs. ghosts: dict storing last known side and position of vanished tracks with a TTL.
update(tid, centre, label)	Called each frame per tracked object. Returns True if the object has just crossed the line for the first time.
expire_track(tid, centre)	Called when a tracked ID disappears. Saves its state as a ghost for GHOST_FRAMES=8 frames.
update_unconfirmed(centre, label, box)	Called for detections without a track ID. Counts them if their centroid is within the band and their box does not duplicate a previous one.
tick_ghosts()	Decrements ghost TTLs and removes expired entries. Called once per frame.

total_counts()	Returns a dict of label -> count combining tracked and unconfirmed counts.
-----------------------	--

2.3 ZoneCounter

Counts unique tracked objects that enter a rectangular zone. Used in the first Surveillance Mode iteration.

Input	x1, y1, x2, y2: pixel coordinates defining the zone rectangle.
update(tid, centre, label)	Counts an object only on the frame it first enters the zone (transition from outside to inside). Suppresses duplicates within DIST_THRESH=120 px using position memory with TTL=80 frames.
total_counts()	Returns label -> count dict for all objects that entered the zone.

2.4 WeaponPersonAssigner

Links weapon detections to the nearest tracked person and raises an ARMED alert.

Input	weapon_boxes: list of (box, conf) tuples from YOLOv8. person_tracks: list of (tid, centre, box) tuples from RT-DETR.
assign(...)	For each weapon detection, finds the person track with the smallest Euclidean centroid distance. If within max_dist=250 px, marks that person as armed. Issues one alert per unique person ID. Draws the weapon box in red and annotates the armed person with a highlighted border and 'ARMED' label.
Output	Modifies frame in-place. Appends alert strings to frame_alerts and global_alerts.

2.5 ShelfTheftDetector

Monitors a shelf zone for items disappearing after a nearby person interaction.

Input	x1, y1, x2, y2: shelf zone coordinates. Fractions of frame size are used: x1=10%, x2=90%, y1=20%, y2=80%.
update_items(item_detections)	Maintains a registry of items (bags, bottles, cups, books) detected within the shelf zone. Items not seen for ITEM_DISAPPEAR_FRAMES=15 frames are removed from registry.
detect_theft(frame, persons, items)	For each person within PICKUP_DIST_PX=120 px of an item, records the association. If the associated item later disappears from the registry, a THEFT ALERT is raised for that person ID (one alert per person).
Output	Returns a list of alert strings. Draws 'THEFT' text on frame if triggered.

2.6 process_traffic(video_path)

The Traffic Mode processing pipeline. Reads frames, runs RT-DETR with ByteTrack on TRAFFIC_CLASSES=[1,2,3,5,7] (bicycle, car, motorbike, bus, truck), estimates per-vehicle speed, counts crossings, and writes an annotated output video.

Input	video_path: str. Path to input video file.
Returns	Tuple of (output_video_path: str, report_text: str).
Speed logic	dist_px = Euclidean distance between current and previous centroid. kmh = dist_px * REAL_WORLD_SCALE * fps * 3.6. Smoothed over 7 frames. Alert fires once per vehicle ID exceeding 90 km/h.
Output video	Annotated frames at original fps written to a temp .mp4 file.

2.7 process_security(video_path)

The Surveillance Mode processing pipeline. Runs RT-DETR on SECURITY_CLASSES=[0,1,2,3,5,7,24,26,28] (persons, vehicles, bags), runs YOLOv8 weapon model every 5 frames, and activates WeaponPersonAssigner and ShelfTheftDetector.

Input	video_path: str. Path to input video file.
Returns	Tuple of (output_video_path: str, report_text: str).
Weapon cadence	YOLOv8 runs on every 5th frame (frame_count % 5 == 0) to reduce latency.
Output video	Annotated frames with weapon boxes, armed person highlights, shelf zone, and item markers.

2.8 run_analysis(video, mode) and Gradio UI

run_analysis() is the Gradio callback. It checks the mode string and dispatches to process_traffic() or process_security(). The Gradio Blocks interface provides a video upload widget, a radio selector for mode, a run button, and side-by-side video and report text outputs.

2.9 Critical Code Snippets

Confidence Threshold Auto-Selection

This logic runs after training to derive the optimal weapon detection threshold by maximizing F1 score:

```
metrics = WEAPON_MODEL.val(data=YAML_PATH, verbose=False)
p_curve = metrics.box.p_curve
r_curve = metrics.box.r_curve

p_mean = p_curve.mean(0)
```

```

r_mean = r_curve.mean(0)
fl_arr = 2 * p_mean * r_mean / (p_mean + r_mean + 1e-9)

best_idx = int(np.argmax(fl_arr))
WEAPON_CONF = float(np.linspace(0, 1, len(fl_arr))[best_idx])
WEAPON_CONF = max(0.30, min(0.85, WEAPON_CONF)) # clamp to sane range

```

Line Crossing Detection (Side-Change Logic)

Core logic in `RobustLineCrossingCounter.update()` that detects when a track crosses the line:

```

def update(self, tid, centre, label):
    side = self._side(centre) # returns +1 or -1

    if tid in self.counted_ids:
        self.prev_side[tid] = side
        return False # already counted

    if tid not in self.prev_side:
        self.prev_side[tid] = side
        return False # first observation, no crossing yet

    prev = self.prev_side[tid]
    crossed = (prev != side) # True if side changed

    if crossed:
        self.counted_ids.add(tid)
        self.labels[tid] = label
        self.prev_side[tid] = side
        return True

    self.prev_side[tid] = side
    return False

```

Weapon-to-Person Assignment

Core loop in `WeaponPersonAssigner.assign()` that links weapons to persons by proximity:

```

for w_box, w_conf in weapon_boxes:
    w_centre = box_center(w_box)
    nearest_tid, nearest_dist = None, float('inf')

    for tid, p_centre, p_box in person_tracks:
        dist = np.hypot(w_centre[0]-p_centre[0], w_centre[1]-p_centre[1])
        if dist < nearest_dist:

```



```
        nearest_dist = dist
        nearest_tid = tid

    if nearest_tid is None or nearest_dist > self.max_dist:
        continue # no person close enough

    self.current_armed_ids.add(nearest_tid)
    if nearest_tid not in self.alerted_ids:
        self.alerted_ids.add(nearest_tid)
        global_alerts.add('ARMED: person carrying a weapon')
```

3. Environment and Reproducibility

3.1 Platform

The project is designed to run in Google Colab with GPU acceleration. It uses Google Drive for persistent model weight storage. It will also run in any environment with a CUDA-capable GPU and the dependencies below installed.

3.2 Dependencies (requirements.txt)

```
# Core ML and Vision
ultralytics>=8.0.0
torch>=2.0.0
torchvision>=0.15.0

# Computer Vision
opencv-python-headless>=4.8.0
numpy>=1.24.0

# Tracking
deep-sort-realtime>=1.3.2

# UI
gradio>=4.0.0

# Model Hub
huggingface-hub>=0.20.0
transformers>=4.40.0

# Archive extraction
rarfile>=4.0
```

3.3 Installation Guide

Step 1. Open the notebook in Google Colab and connect to a GPU runtime (Runtime > Change runtime type > T4 GPU or better).

Step 2. Run Cell 1 to install all dependencies:

```
!pip install -q ultralytics deep-sort-realtime opencv-python-headless gradio rarfile
!pip install -U transformers
```

Step 3. Mount Google Drive and verify paths:

```
from google.colab import drive
drive.mount('/content/drive')
# Ensure your Drive has: /MyDrive/WeaponDetectionProject/weapon_model/weights/best.pt
```

Step 4. Upload the labeled gun dataset ZIP to /content/Gun_data_labeled.zip and run the dataset extraction cell (Cell 2).

Step 5. Run the training cell (Cell 3) to train or resume training the YOLOv8 weapon model.

Step 6. Run the model loading cell (Cell 4) to load WEAPON_MODEL (YOLOv8) and GENERAL_MODEL (RT-DETR).

Step 7. Run the confidence threshold cell (Cell 5) to auto-select WEAPON_CONF and GENERAL_CONF.

Step 8. Run the final Gradio cell (the last cell) to launch the web interface. A public share URL is printed. Upload a video, select a mode, and click Run Analysis.

3.4 Environment Variables and Configuration

There are no .env files or secret API keys required. All configuration is set as Python constants at the top of the main processing cell:

PROJECT_PATH	Path to the Google Drive project folder. Default: /content/drive/MyDrive/WeaponDetectionProject
BEST_WEIGHTS	Path to trained YOLOv8 weights. Default: PROJECT_PATH/weapon_model/weights/best.pt
SPEED_LIMIT_KMH	Speed threshold for alerts in Traffic Mode. Default: 90
LINE_FRAC	Vertical fraction for the counting line. Default: 0.50 (mid-frame)
LINE_BAND_HALF	Half-width of the counting band in pixels. Default: 30
GHOST_FRAMES	Frames to retain ghost track memory. Default: 8
BASE_PX_PER_METER	Pixel-to-meter scale for speed estimation. Default: 8.0
WEAPON_ASSIGN_DIST	Max pixel distance for weapon-to-person assignment. Default: 180
SHELF_X1_FRAC .. SHELF_Y2_FRAC	Shelf zone boundaries as fractions of frame size. Default: x[0.10,0.90], y[0.20,0.80]
ITEM_DISAPPEAR_FRAMES	Frames before a missing item is considered gone. Default: 15
PICKUP_DIST_PX	Pixel distance threshold for person-item proximity. Default: 120
GENERAL_CONF	RT-DETR confidence threshold. Auto-selected or default 0.60
WEAPON_CONF	YOLOv8 confidence threshold. Auto-selected or default 0.40

3.5 Training Configuration

Base model	yolov8m.pt (pretrained on COCO)
Dataset	Custom gun dataset (1 class: weapon)
Epochs	50 (with save_period=5)
Image size	640 x 640
Batch size	16
Device	GPU (CUDA 0) if available, else CPU
Resume	Automatic if last.pt exists in weights directory

3.6 Alternernate Weapon Model (HuggingFace)

A second notebook version loads a pre-trained weapon detection model from HuggingFace Hub instead of training from scratch. This is used when Drive weights are unavailable:

```
from huggingface_hub import hf_hub_download
MODEL_REPO = 'Subh775/Threat-Detection-YOLOv8n'
model_path = hf_hub_download(repo_id=MODEL_REPO, filename='weights/best.pt')
WEAPON_MODEL = YOLO(model_path)
WEAPON_CONF = 0.40
```

Appendix: Class and Function Reference

Class / Function	Module / Role
RobustLineCrossingCounter	Traffic and Surveillance: counts unique objects crossing a line band with ghost memory
ZoneCounter	Surveillance (v1): counts unique objects entering a rectangular zone
WeaponPersonAssigner	Surveillance: assigns weapon detections to nearest person and fires ARMED alert
ShelfTheftDetector	Surveillance: monitors shelf zone items and fires THEFT ALERT on disappearance
get_label(cls)	Util: maps COCO class int to label string
box_center(box)	Util: returns centroid of bounding box
iou(b1, b2)	Util: computes IoU between two boxes
fuse_detections(dets)	Util: NMS over weapon detections
draw_box(...)	Util: draws labeled bounding box on frame
draw_box_with_speed(...)	Util: draws labeled bounding box with speed annotation
overlay_alerts(frame, lines)	Util: renders alert text onto frame
draw_counts(frame, counts)	Util: renders object counts onto frame
process_traffic(video_path)	Pipeline: traffic mode full processing loop
process_security(video_path)	Pipeline: surveillance mode full processing loop
run_analysis(video, mode)	Dispatcher: Gradio callback routing to correct pipeline
find_dataset_root(base)	Setup: locates images/labels root inside extracted dataset